

UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Minas - Sede Medellin

Programacion Orientada a Objetos 2026-1

Actividad 5 - En equipo (20%)

Aplicacion de Interfaz Grafica con CRUD sobre archivo

Docente: Walter Hugo Arboleda Mazo

Integrantes:

Ivan Camilo Gomez Gallego - 1092853135

Juan Andres Builes Arenas - 1021924317

Fecha de entrega: Jueves 9 de julio de 2026

Repositorio GitHub:

<https://github.com/ivannk12/poo-actividad5>

Descripcion de la aplicacion

Enunciado:

Aplicacion de Interfaz Grafica de Usuario (Swing) que implementa un formulario con los botones Create, Read, Update y Delete (CRUD). La informacion se persiste en un archivo de texto (estudiantes.txt), siguiendo el manejo de archivos en Java (FileWriter, PrintWriter, FileReader y BufferedReader). Como caso de uso se administra un registro de estudiantes, donde cada estudiante posee un identificador, un nombre, un programa academico y una edad.

Operaciones CRUD implementadas:

- **Create (Crear):** agrega un nuevo estudiante al final del archivo.
- **Read (Leer):** lee todos los estudiantes del archivo y los muestra en una tabla.
- **Update (Actualizar):** modifica los datos de un estudiante identificado por su ID.
- **Delete (Eliminar):** elimina del archivo el estudiante identificado por su ID.

Arquitectura de clases:

- **Estudiante:** modelo de datos; convierte el objeto a/desde una linea de texto.
- **GestorArchivo:** concentra la logica CRUD de lectura y escritura del archivo.
- **VentanaPrincipal:** interfaz grafica (formulario, botones y tabla de resultados).
- **Principal:** punto de entrada de la aplicacion.

Material guia: file-handling in Java with CRUD operations (GeeksforGeeks).

Interfaz grafica de usuario:

The screenshot shows a Java Swing window with a light gray background. At the top, there's a black header bar. Below it, the form has four input fields: 'ID:' with the value '4', 'Programa:' with 'Ingenieria de Sistemas', 'Nombre:' with 'Juan Builes', and 'Edad:' with '21'. Below these fields are five buttons: 'Crear', 'Leer', 'Actualizar', 'Eliminar', and 'Limpiar'. A green message 'Se leyeron los estudiantes del archivo.' is displayed above a table. The table has four columns: 'ID', 'Nombre', 'Programa', and 'Edad'. It contains three rows of data.

ID	Nombre	Programa	Edad
1	Ana Torres	Ingenieria de Sistemas	20
2	Carlos Ruiz	Ingenieria Industrial	22
3	Maria Gomez	Ingenieria de Minas	19

Diagrama de clases (UML)

Diagrama de clases - Actividad 5 (CRUD de Estudiantes sobre archivo)

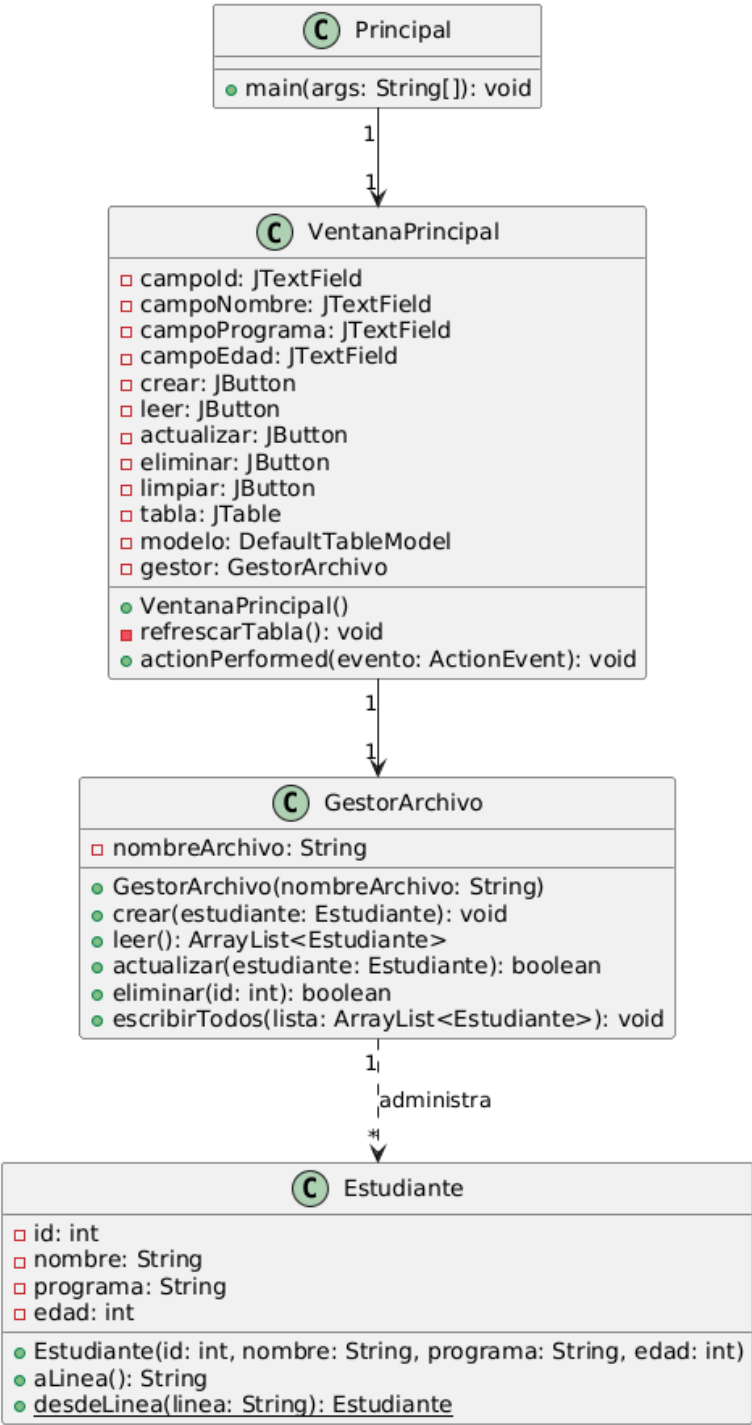
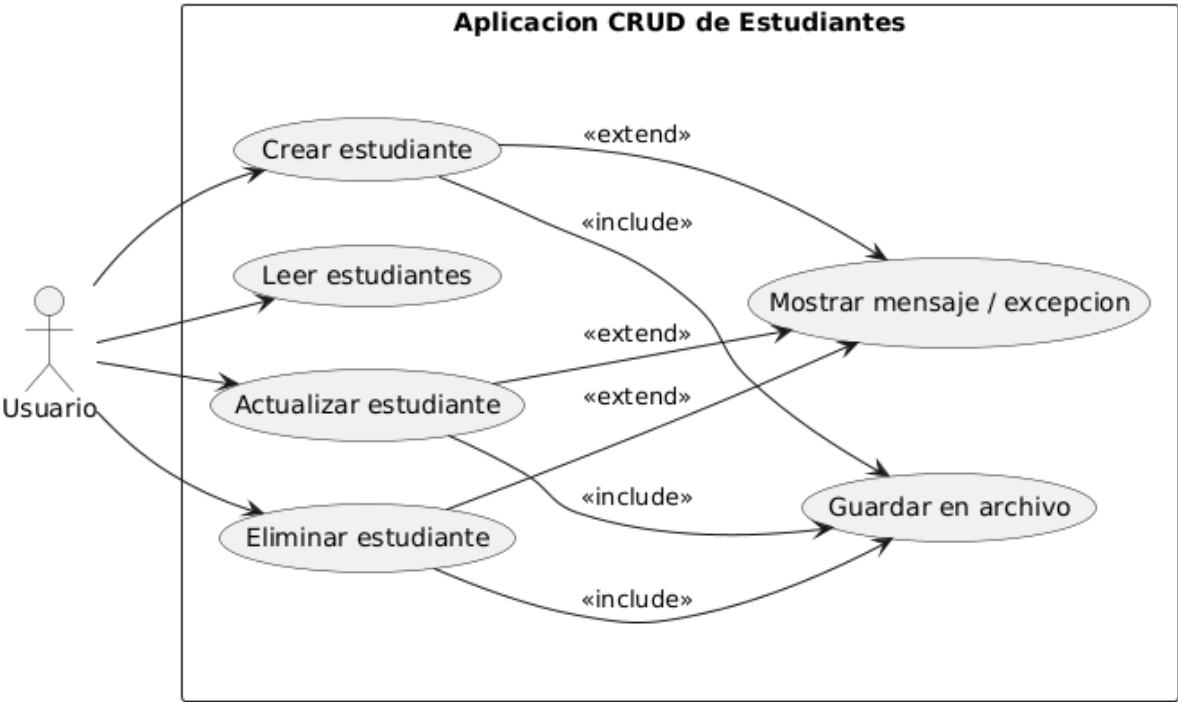


Diagrama de casos de uso (UML)

Casos de uso - Actividad 5 (CRUD de Estudiantes sobre archivo)



Codigo fuente (Java)

Estudiante.java

```
// Clase Estudiante: guarda los datos de un estudiante (id, nombre, programa y edad).
// Tambien pasa el objeto a texto para el archivo y lo vuelve a armar desde el archivo.
public class Estudiante {

    int id;
    String nombre;
    String programa;
    int edad;

    Estudiante(int id, String nombre, String programa, int edad) {
        this.id = id;
        this.nombre = nombre;
        this.programa = programa;
        this.edad = edad;
    }

    // Pasa el estudiante a una linea separada por comas para guardarlo en el archivo
    String aLinea() {
        return id + "," + nombre + "," + programa + "," + edad;
    }

    // Arma un estudiante a partir de una linea leida del archivo
    static Estudiante desdeLinea(String linea) {
        String[] datos = linea.split(",");
        int id = Integer.parseInt(datos[0]);
        String nombre = datos[1];
        String programa = datos[2];
        int edad = Integer.parseInt(datos[3]);
        return new Estudiante(id, nombre, programa, edad);
    }
}
```

<https://github.com/ivannk12/poo-actividad5/blob/main/app/Estudiante.java>

GestorArchivo.java

```
import java.io.*;
import java.util.ArrayList;

// Clase que maneja el archivo de estudiantes con las operaciones CRUD.
// Para escribir usa FileWriter y PrintWriter, y para leer FileReader con BufferedReader.
public class GestorArchivo {

    String nombreArchivo;

    GestorArchivo(String nombreArchivo) {
        this.nombreArchivo = nombreArchivo;
    }

    // CREATE: agrega un estudiante al final del archivo
    void crear(Estudiante estudiante) throws IOException {
        FileWriter archivo = new FileWriter(nombreArchivo, true); // true = agrega al final
        PrintWriter impresor = new PrintWriter(archivo);
        impresor.println(estudiante.aLinea());
        impresor.close();
    }

    // READ: lee todos los estudiantes del archivo y los devuelve en una lista
    ArrayList<Estudiante> leer() throws IOException {
        ArrayList<Estudiante> lista = new ArrayList<Estudiante>();
        File archivo = new File(nombreArchivo);
        if (!archivo.exists()) {
            return lista; // si todavia no existe el archivo, la lista queda vacia
        }
    }
}
```

```

    }
    BufferedReader filtro = new BufferedReader(new FileReader(nombreArchivo));
    String linea = filtro.readLine();
    while (linea != null) {
        if (!linea.trim().equals("")) {
            lista.add(Estudiante.desdeLinea(linea));
        }
        linea = filtro.readLine();
    }
    filtro.close();
    return lista;
}

// UPDATE: busca por id, cambia los datos y reescribe el archivo
boolean actualizar(Estudiante estudiante) throws IOException {
    ArrayList<Estudiante> lista = leer();
    boolean encontrado = false;
    for (int i = 0; i < lista.size(); i++) {
        if (lista.get(i).id == estudiante.id) {
            lista.set(i, estudiante);
            encontrado = true;
        }
    }
    if (encontrado) {
        escribirTodos(lista);
    }
    return encontrado;
}

// DELETE: quita el estudiante con ese id y reescribe el archivo
boolean eliminar(int id) throws IOException {
    ArrayList<Estudiante> lista = leer();
    ArrayList<Estudiante> nueva = new ArrayList<Estudiante>();
    boolean encontrado = false;
    for (int i = 0; i < lista.size(); i++) {
        if (lista.get(i).id == id) {
            encontrado = true; // este no se copia a la nueva lista
        } else {
            nueva.add(lista.get(i));
        }
    }
    if (encontrado) {
        escribirTodos(nueva);
    }
    return encontrado;
}

// Reescribe todo el archivo con la lista que se le pase
void escribirTodos(ArrayList<Estudiante> lista) throws IOException {
    FileWriter archivo = new FileWriter(nombreArchivo, false); // false = sobrescribe
    PrintWriter impresor = new PrintWriter(archivo);
    for (int i = 0; i < lista.size(); i++) {
        impresor.println(lista.get(i).aLinea());
    }
    impresor.close();
}
}

```

<https://github.com/ivannk12/poo-actividad5/blob/main/app/GestorArchivo.java>

VentanaPrincipal.java

```

import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;
import java.util.ArrayList;

```

```

import javax.swing.*;
import javax.swing.event.ListSelectionEvent;
import javax.swing.event.ListSelectionListener;
import javax.swing.table.DefaultTableModel;

// Ventana de la aplicacion: formulario con los botones Crear, Leer, Actualizar y
// Eliminar (CRUD) que trabajan sobre el archivo por medio de GestorArchivo.
// Los estudiantes se muestran en una tabla.
public class VentanaPrincipal extends JFrame implements ActionListener, ListSelectionListener {

    Container contenedor;
    JLabel etId, etNombre, etPrograma, etEdad, mensaje;
    JTextField campoId, campoNombre, campoPrograma, campoEdad;
    JButton crear, leer, actualizar, eliminar, limpiar;
    JTable tabla;
    DefaultTableModel modelo;
    JScrollPane scroll;

    GestorArchivo gestor;

    public VentanaPrincipal() {
        gestor = new GestorArchivo("estudiantes.txt");
        inicio();
        setTitle("CRUD de Estudiantes - Archivo");
        setSize(640, 470);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
    }

    private void inicio() {
        contenedor = getContentPane();
        contenedor.setLayout(null);

        etId = new JLabel("ID:");
        etId.setBounds(20, 20, 80, 23);
        campoId = new JTextField();
        campoId.setBounds(100, 20, 180, 23);

        etNombre = new JLabel("Nombre:");
        etNombre.setBounds(20, 50, 80, 23);
        campoNombre = new JTextField();
        campoNombre.setBounds(100, 50, 180, 23);

        etPrograma = new JLabel("Programa:");
        etPrograma.setBounds(300, 20, 80, 23);
        campoPrograma = new JTextField();
        campoPrograma.setBounds(390, 20, 220, 23);

        etEdad = new JLabel("Edad:");
        etEdad.setBounds(300, 50, 80, 23);
        campoEdad = new JTextField();
        campoEdad.setBounds(390, 50, 220, 23);

        crear = new JButton("Crear");
        crear.setBounds(20, 90, 110, 28);
        crear.addActionListener(this);

        leer = new JButton("Leer");
        leer.setBounds(140, 90, 110, 28);
        leer.addActionListener(this);

        actualizar = new JButton("Actualizar");
        actualizar.setBounds(260, 90, 110, 28);
        actualizar.addActionListener(this);

        eliminar = new JButton("Eliminar");

```

```

        eliminar.setBounds(380, 90, 110, 28);
        eliminar.addActionListener(this);

        limpiar = new JButton("Limpiar");
        limpiar.setBounds(500, 90, 110, 28);
        limpiar.addActionListener(this);

        mensaje = new JLabel(" ");
        mensaje.setForeground(new Color(0, 100, 0));
        mensaje.setBounds(20, 125, 590, 23);

        // Tabla donde se muestran los estudiantes
        modelo = new DefaultTableModel();
        modelo.addColumn("ID");
        modelo.addColumn("Nombre");
        modelo.addColumn("Programa");
        modelo.addColumn("Edad");
        tabla = new JTable(modelo);
        scroll = new JScrollPane(tabla);
        scroll.setBounds(20, 155, 590, 265);

        // Al seleccionar una fila se cargan sus datos en el formulario
        tabla.getSelectionModel().addListSelectionListener(this);

        contenedor.add(etId);
        contenedor.add(campoId);
        contenedor.add(etNombre);
        contenedor.add(campoNombre);
        contenedor.add(etPrograma);
        contenedor.add(campoPrograma);
        contenedor.add(etEdad);
        contenedor.add(campoEdad);
        contenedor.add(crear);
        contenedor.add(leer);
        contenedor.add(actualizar);
        contenedor.add(eliminar);
        contenedor.add(limpiar);
        contenedor.add(mensaje);
        contenedor.add(scroll);
    }

    // Cuando el usuario selecciona una fila, pasa esos datos al formulario
    @Override
    public void valueChanged(ListSelectionEvent evento) {
        int fila = tabla.getSelectedRow();
        if (fila >= 0) {
            campoId.setText(modelo.getValueAt(fila, 0).toString());
            campoNombre.setText(modelo.getValueAt(fila, 1).toString());
            campoPrograma.setText(modelo.getValueAt(fila, 2).toString());
            campoEdad.setText(modelo.getValueAt(fila, 3).toString());
        }
    }

    // Vuelve a llenar la tabla con lo que hay en el archivo
    private void refrescarTabla() throws IOException {
        modelo.setRowCount(0);
        ArrayList<Estudiante> lista = gestor.leer();
        for (int i = 0; i < lista.size(); i++) {
            Estudiante e = lista.get(i);
            modelo.addRow(new Object[]{e.id, e.nombre, e.programa, e.edad});
        }
    }

    private void limpiarCampos() {
        campoId.setText("");
        campoNombre.setText("");
        campoPrograma.setText("");
    }

```

```

        campoEdad.setText("");
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        try {
            mensaje.setForeground(new Color(0, 100, 0));
            if (evento.getSource() == crear) {
                Estudiante e = new Estudiante(
                    Integer.parseInt(campoId.getText()),
                    campoNombre.getText(),
                    campoPrograma.getText(),
                    Integer.parseInt(campoEdad.getText()));
                gestor.crear(e);
                refrescarTabla();
                mensaje.setText("Estudiante creado correctamente.");
                limpiarCampos();
            } else if (evento.getSource() == leer) {
                refrescarTabla();
                mensaje.setText("Se leyeron los estudiantes del archivo.");
            } else if (evento.getSource() == actualizar) {
                Estudiante e = new Estudiante(
                    Integer.parseInt(campoId.getText()),
                    campoNombre.getText(),
                    campoPrograma.getText(),
                    Integer.parseInt(campoEdad.getText()));
                if (gestor.actualizar(e)) {
                    mensaje.setText("Estudiante actualizado correctamente.");
                } else {
                    mensaje.setText("No se encontro un estudiante con ese ID.");
                }
                refrescarTabla();
                limpiarCampos();
            } else if (evento.getSource() == eliminar) {
                int id = Integer.parseInt(campoId.getText());
                if (gestor.eliminar(id)) {
                    mensaje.setText("Estudiante eliminado correctamente.");
                } else {
                    mensaje.setText("No se encontro un estudiante con ese ID.");
                }
                refrescarTabla();
                limpiarCampos();
            } else if (evento.getSource() == limpiar) {
                limpiarCampos();
                mensaje.setText(" ");
            }
        } catch (NumberFormatException e) {
            mensaje.setForeground(Color.RED);
            mensaje.setText("El ID y la edad deben ser numeros enteros.");
        } catch (IOException e) {
            mensaje.setForeground(Color.RED);
            mensaje.setText("Error al acceder al archivo.");
        }
    }
}

```

<https://github.com/ivannk12/poo-actividad5/blob/main/app/VentanaPrincipal.java>

Principal.java

```

// Clase principal: abre la ventana de la aplicacion
public class Principal {

    public static void main(String[] args) {
        VentanaPrincipal miVentana = new VentanaPrincipal();
        miVentana.setVisible(true);
    }
}

```

```
}
```

<https://github.com/ivannk12/poo-actividad5/blob/main/app/Principal.java>